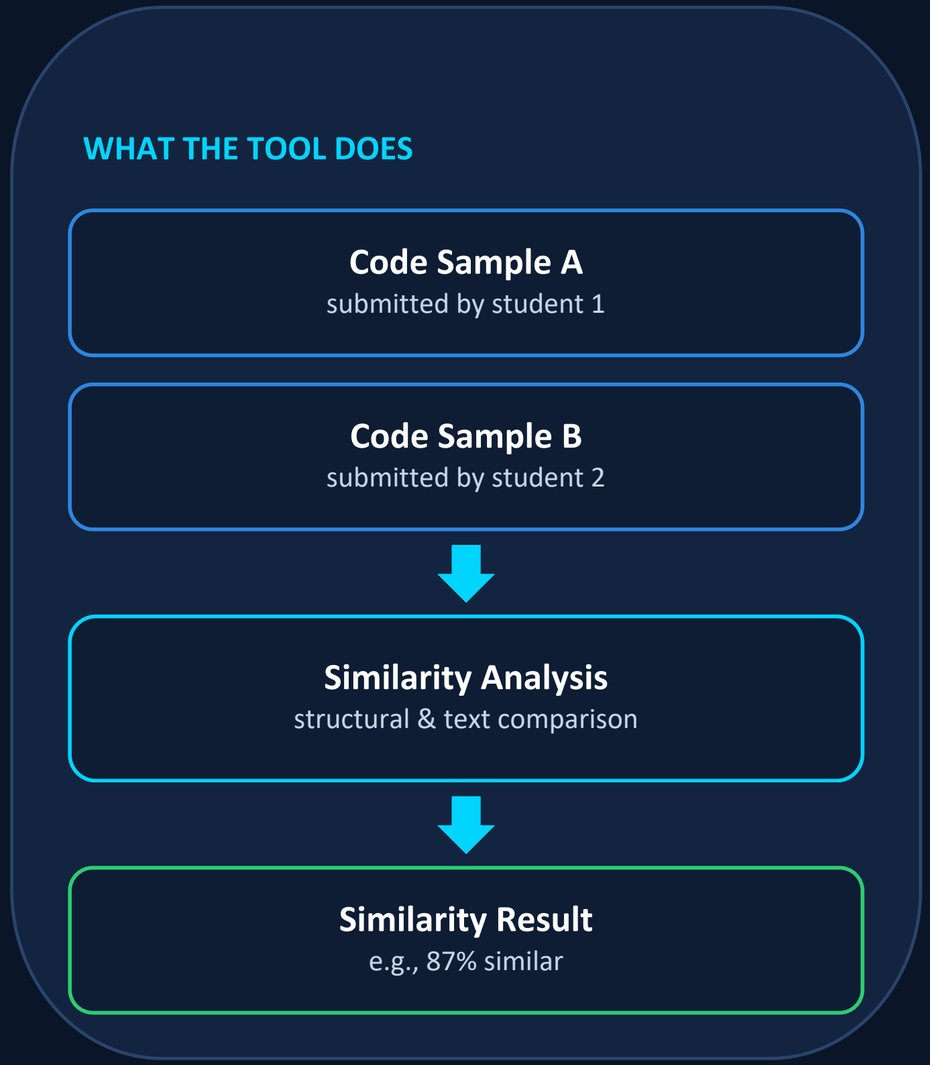
AI-Based Plagiarism Detection & Code Similarity Analyzer

Introduction

- Code plagiarism
  - Submitting or reusing another person's source code without proper attribution.
- Code similarity
  - A measurable degree of resemblance between two code samples, based on structure and content.
- Why automate the comparison?
  - Manual review of many submissions is slow and often inconsistent.
  - An automated tool quickly flags pairs of submissions that need closer human inspection.
- A similarity score is an indicator for review — not proof of plagiarism.



Problem Statement

- Manually comparing multiple programming assignments is time-consuming and error-prone.
- With large class sizes, line-by-line review of every submission pair is impractical.
- Superficial edits — renamed variables, changed comments, reformatted code — can hide copying from manual review.
- Reviewers need a fast first filter that highlights only the submission pairs worth a closer look.

THE NEED

A simple application that accepts two code samples and displays a clear comparison result.

WHO BENEFITS

- **Faculty & teaching assistants**
- **Code reviewers & mentors**
- **Students learning good practice**

Objectives

1	**Clear Input Interface**	Provide a web interface to enter or submit two code samples.
2	**Frontend ↔ Backend**	Connect the frontend to a backend analysis service.
3	**Code Comparison**	Compare the two inputs using the similarity algorithm implemented in the project.
4	**Readable Results**	Display the similarity result in an understandable form.
5	**Version Control**	Maintain project code and documentation on GitHub.

Proposed System

- The user opens the web application and supplies two code samples through the frontend.
- The frontend validates the input and forwards it to the backend API.
- The backend runs the similarity logic and returns a result, which the frontend displays.



Response path: Backend API → Frontend → Similarity score or error message shown to the user

The frontend never computes similarity — it only collects input and renders the backend's response, which keeps the analysis logic centralized and consistent.

Technologies Used

FRONTEND

- HTML5 — page structure
- CSS3 — styling & layout
- JavaScript — interaction & API calls

BACKEND

- Python — application logic
- Flask — REST API & request handling
- Similarity analysis module

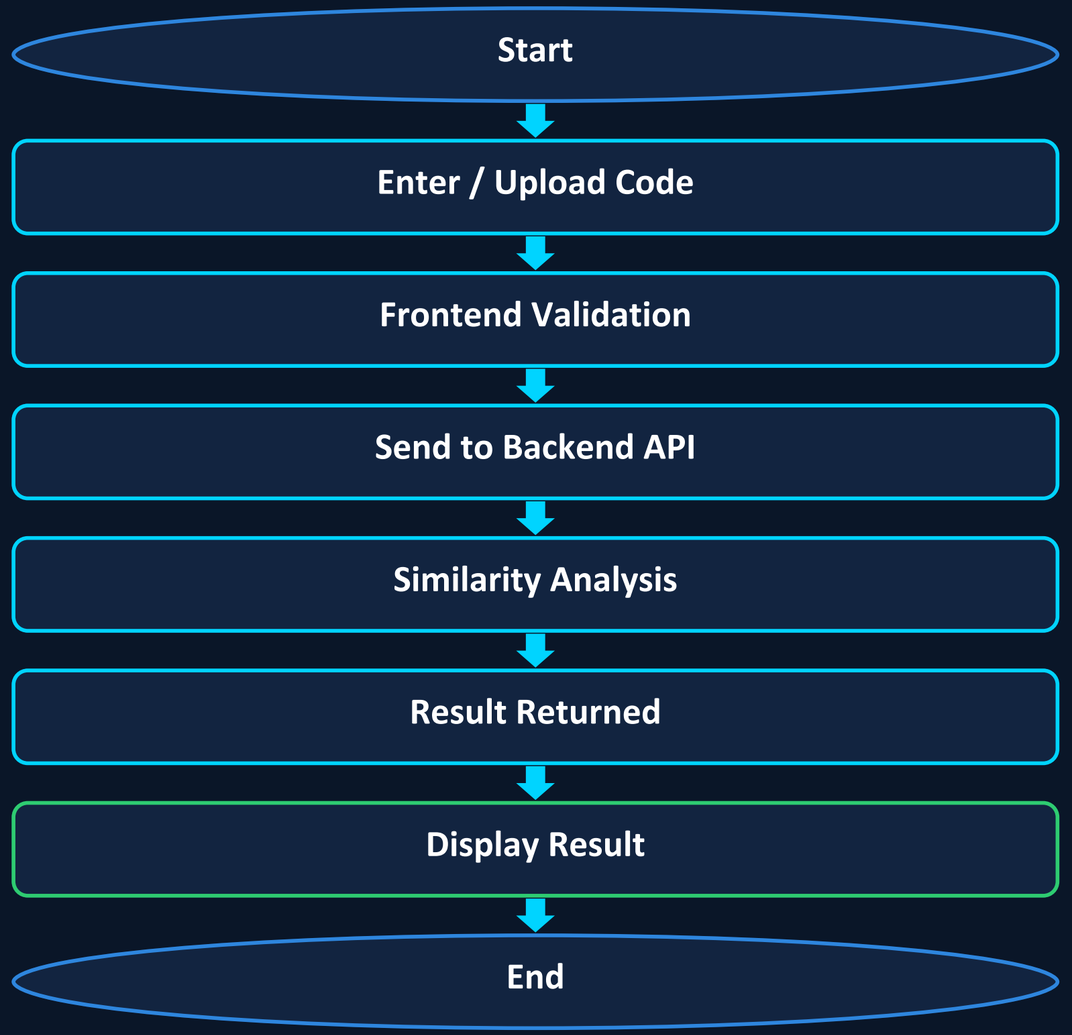
TOOLS

- Visual Studio Code — development
- Git & GitHub — version control
- Web browser & Live Server — local testing

Confirmed from the GitHub repository: backend/ (Flask app) · frontend/ (static site) · docs/

System Workflow

- Enter / upload
  – The user provides two code samples through the web interface.
- Validate
  – The frontend checks that inputs are not empty before sending.
- Analyze
  – The backend receives both samples and runs the similarity comparison.
- Display
  – The result — a similarity score or a clear error message — is shown to the user.



Project Implementation

Frontend Module

Presents input areas for both code samples, action buttons, validation messages, and the result area.

Backend Module

Receives requests, validates the data, invokes the analysis, and returns a JSON response.

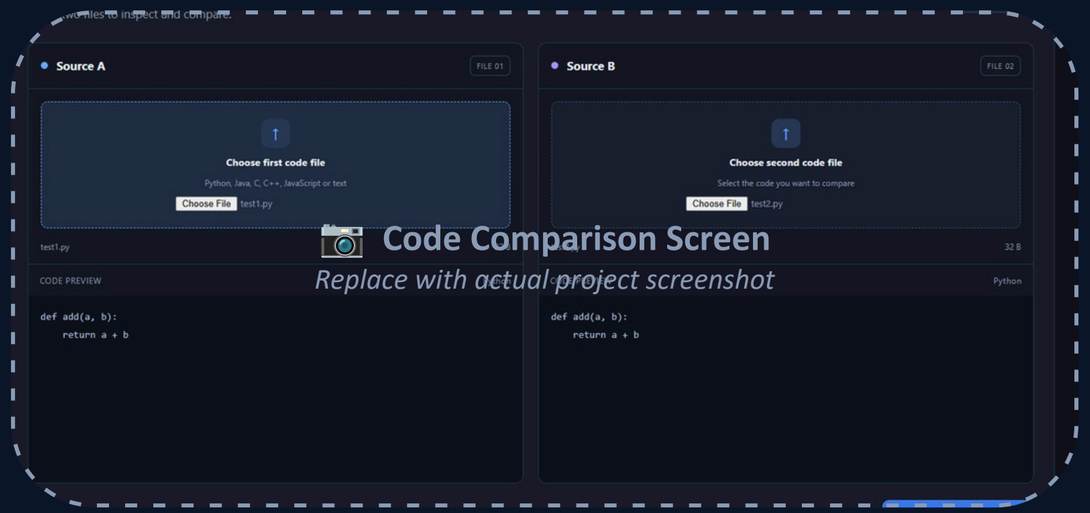
Similarity Module

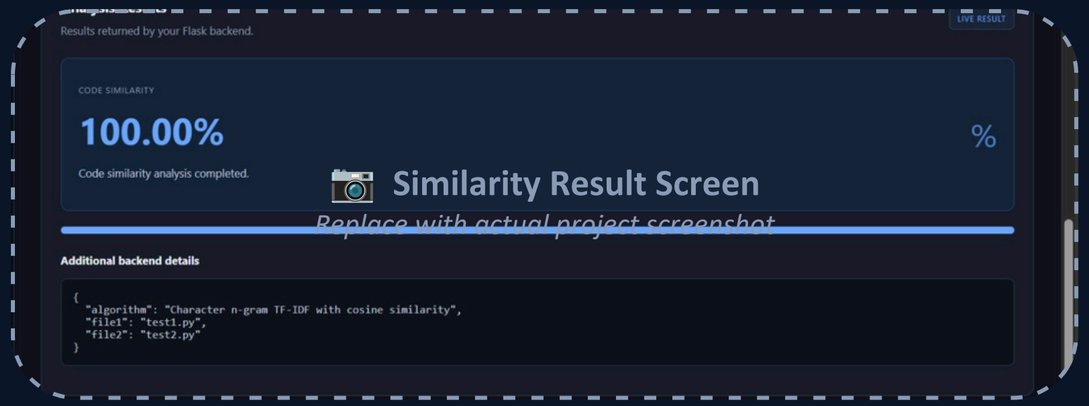
Compares the submitted code samples according to the comparison algorithm implemented in the project.

REPOSITORY STRUCTURE (GitHub)

Code-Similarity-Analyzer/
├─ backend/ → Flask app & similarity analysis
└─ frontend/ → static HTML/CSS/JS interface

Output and Testing





Test case	Input / Action	Expected result
Identical code	Submit the same code twice	High / max similarity (algorithm-dependent)
Different code	Submit unrelated samples	Generally lower similarity
Empty input	Leave one or both inputs empty	Validation message or controlled error
Backend unavailable	Stop backend, then analyze	Clear connection error — no silent failure
Repeat test	Run the same comparison again	Consistent result (deterministic logic)

Advantages and Limitations

✔ ADVANTAGES

- Speeds up the initial review of code submissions.
- Consistent, repeatable comparison on every run.
- Simple interface — usable by non-technical reviewers.
- Supports academic-integrity checks and learning.
- Lightweight stack: easy to deploy, run, and extend.

⚠ LIMITATIONS

- Similar code does not automatically prove plagiarism.
- Common patterns (loops, standard idioms) can raise similarity.
- Accuracy depends on the implemented comparison algorithm.
- Currently supports a limited set of languages / constructs.
- Results always require human interpretation and judgment.

Future Scope



Matching-Line Highlighting

Visually highlight the specific lines that match between the two samples.



More Languages

Extend support beyond the current language(s) to Python, Java, C++, etc.



Smarter Algorithms

Token-based or AST-level comparison, and machine-learning similarity models.



Robustness

Better error handling, input sanitization, and automated test suites.



User Accounts

Saved comparison history and report export for reviewers.



Batch Comparison

Compare one submission against an entire class in a single run.

Conclusion

- Built a working web application that compares two source-code samples and displays a similarity result.
- Connected a static HTML/CSS/JavaScript frontend to a Python + Flask backend API.
- Organized the project into frontend, backend, and similarity modules under GitHub version control.
- Demonstrated practical skills in full-stack integration, testing, and documentation.
- The tool supports faster, fairer code review — with results always interpreted by a person.

THANK YOU!